

Brain Tumor Detection Using Semantic Segmentation



Course: Computer Vision

Group: P18

Name: Ali Davut Eskiocak

E-mail: ali.eskiocak@edu.uah.es

Delivery Date: 19 May 2026

Contents

1- Introduction

2- Development Phase

3- Conclusions

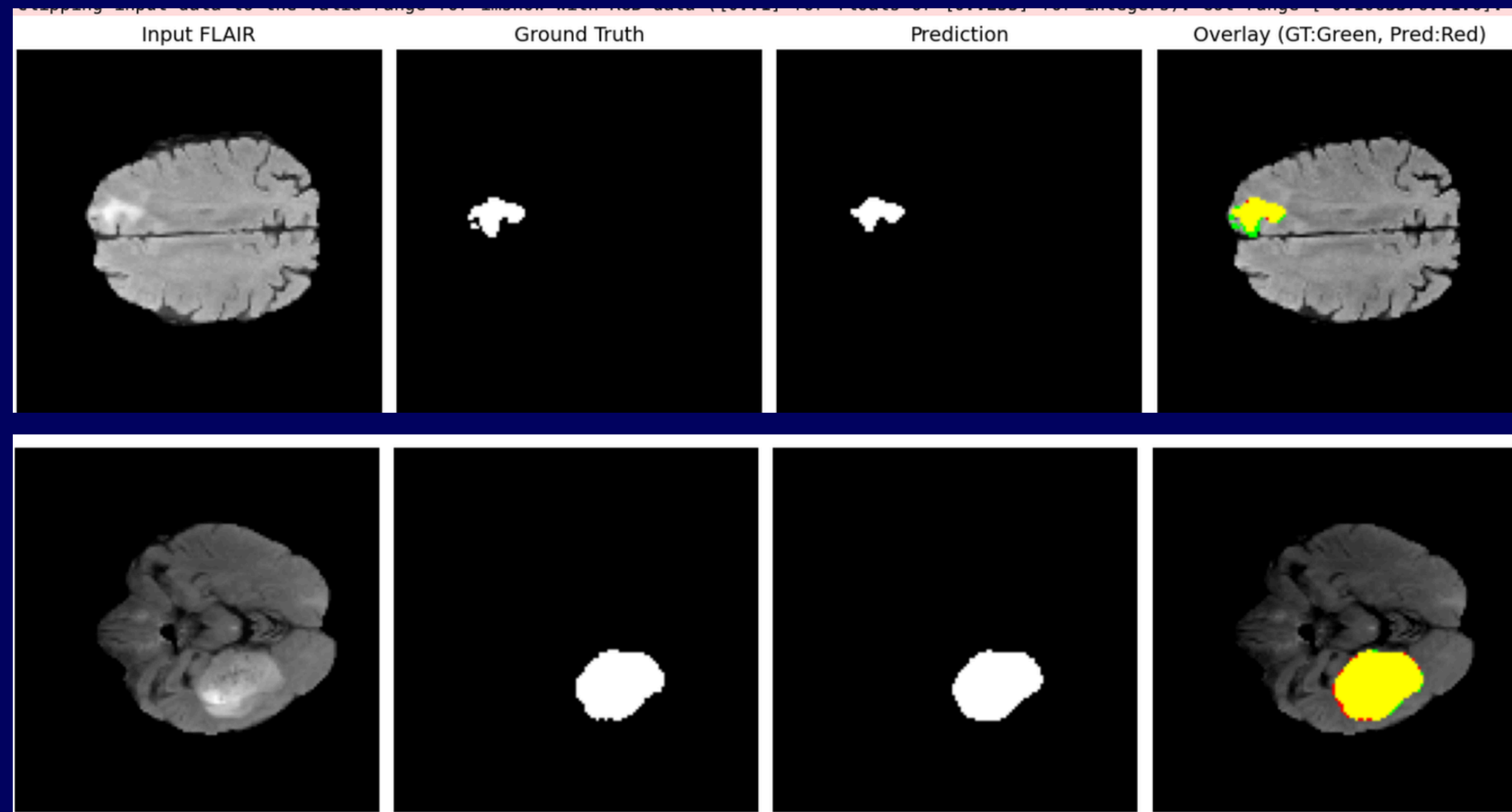
4- Results

5- Sources

Which dataset that I used?

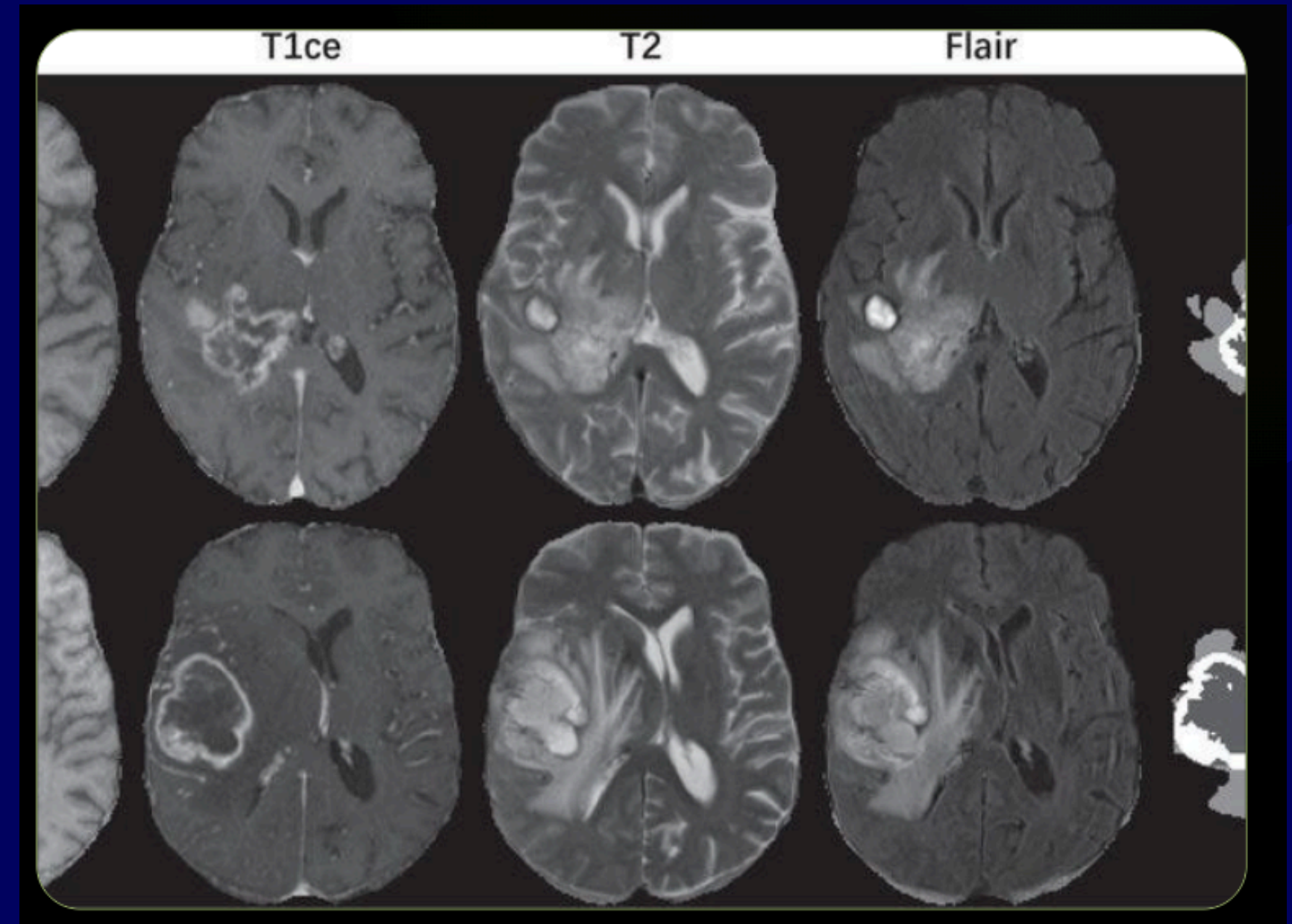
Brain Tumor Segmentation(BraTS2020)

- 369 subjects
- 70-15-15
- Detecting tumors with Pytorch with Semantic Segmentation



Data Engineering & Preprocessing

- **BraTS 2020 Dataset:** High-Grade Glioma (HGG) and Low-Grade Glioma (LGG) slices provided in .h5 format.
- **FLAIR Modality:** Specifically focused on Fluid-Attenuated Inversion Recovery for superior edema and tumor contrast.
- **Binary Mask Consolidation:** Multi-class ground truth labels merged into a unified binary ROI for efficient training.
- **Normalization:** Dynamic Min-Max scaling per slice to ensure consistent intensity range across varying clinical scanners.



Technical Framework



U-Net Architecture

Standard encoder-decoder structure with skip connections. Features 4 down-sampling blocks and bottleneck with 1024 filters for deep feature extraction.



Albumentations

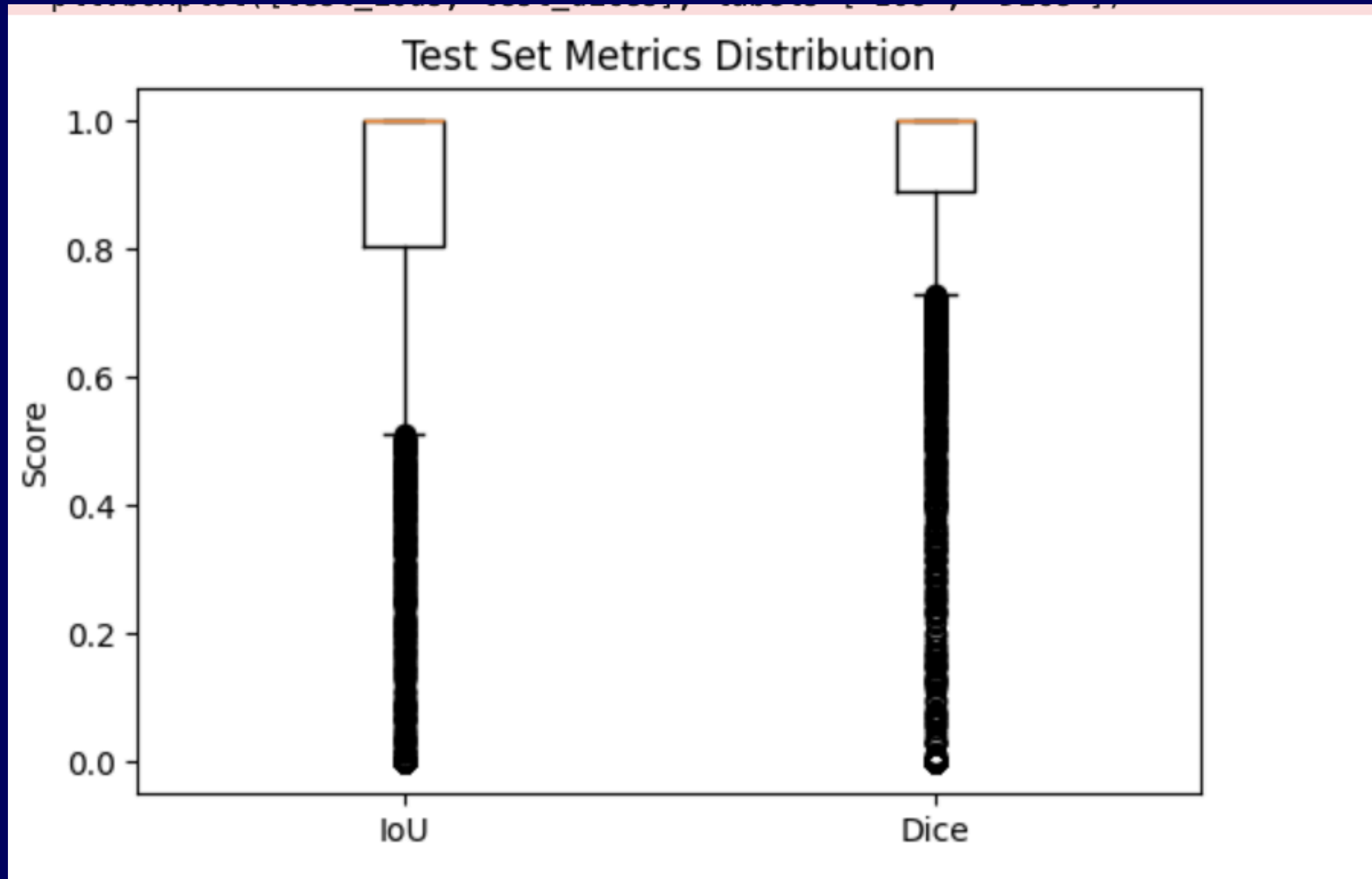
Advanced spatial variance via flips, 90-degree rotations, and shift-scale-rotate transforms to prevent overfitting on localized tumor shapes.



BCEDiceLoss

Compound optimization combining Binary Cross-Entropy for pixel accuracy and Dice Loss to maximize spatial overlap and handle class imbalance.

How Much Achievement that I Succeeded ?

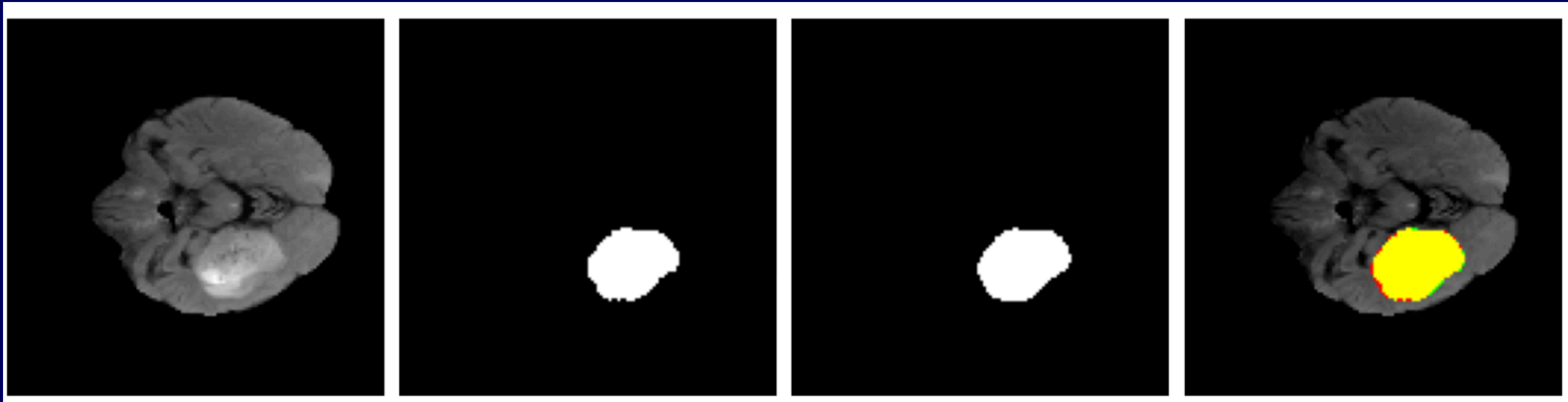


(Version 1e-4 and with augmentation)

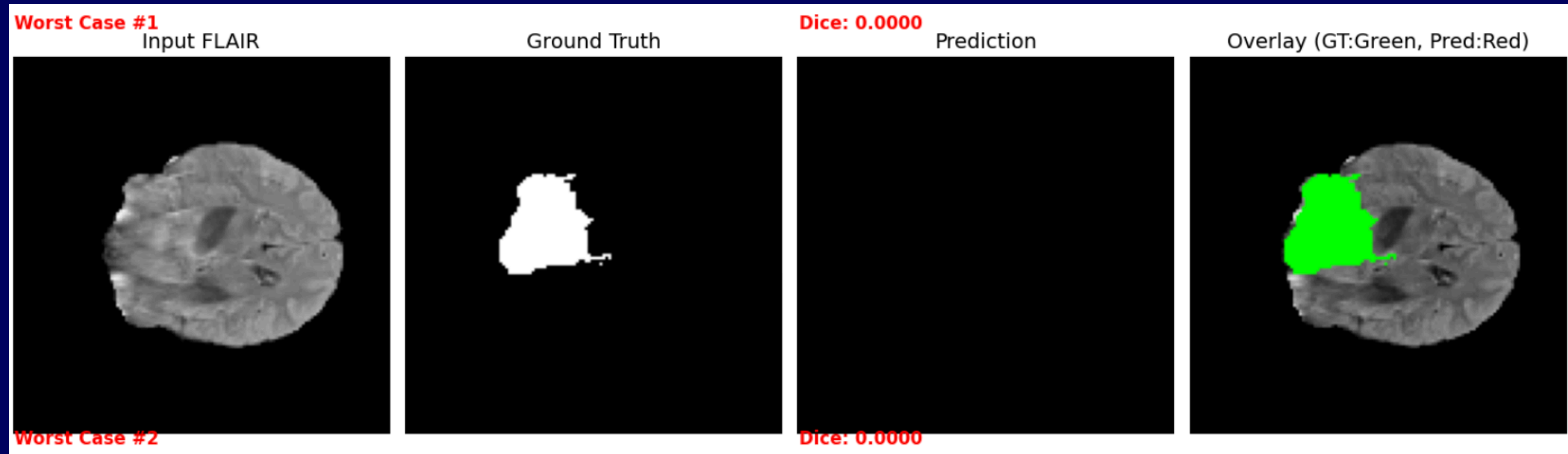
Same kind of graph also drew for the other versions. You can refer the .pynb files.

There is a more detailed analysis in the G18_VA_SVA-EskiocakAliDavut.ipynb file.

Good Example



Bad Example

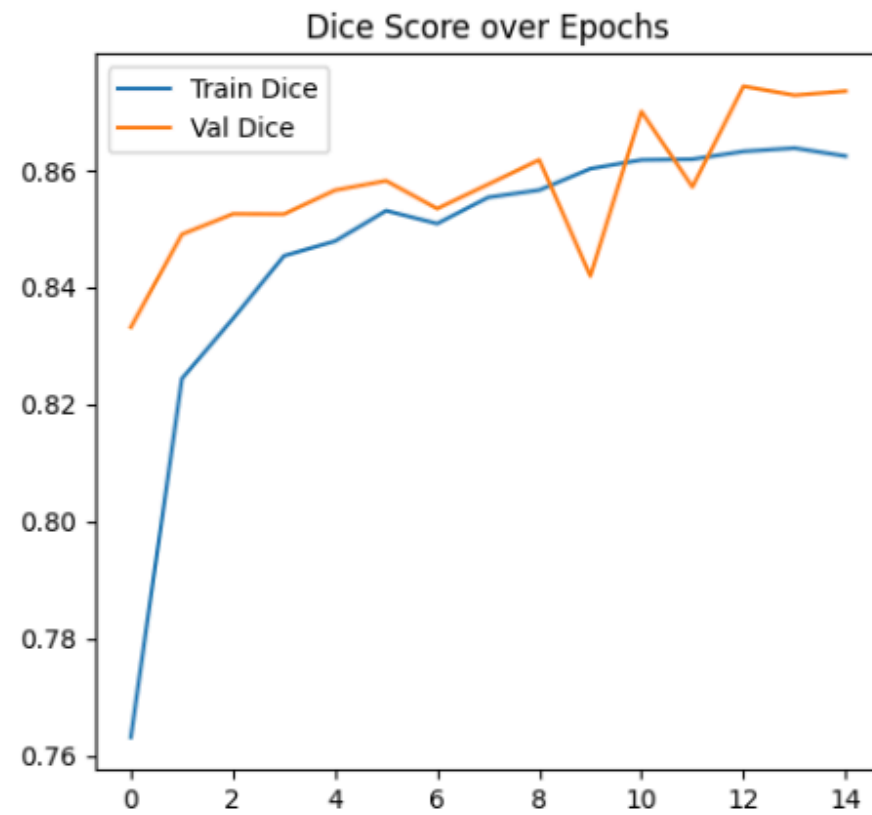
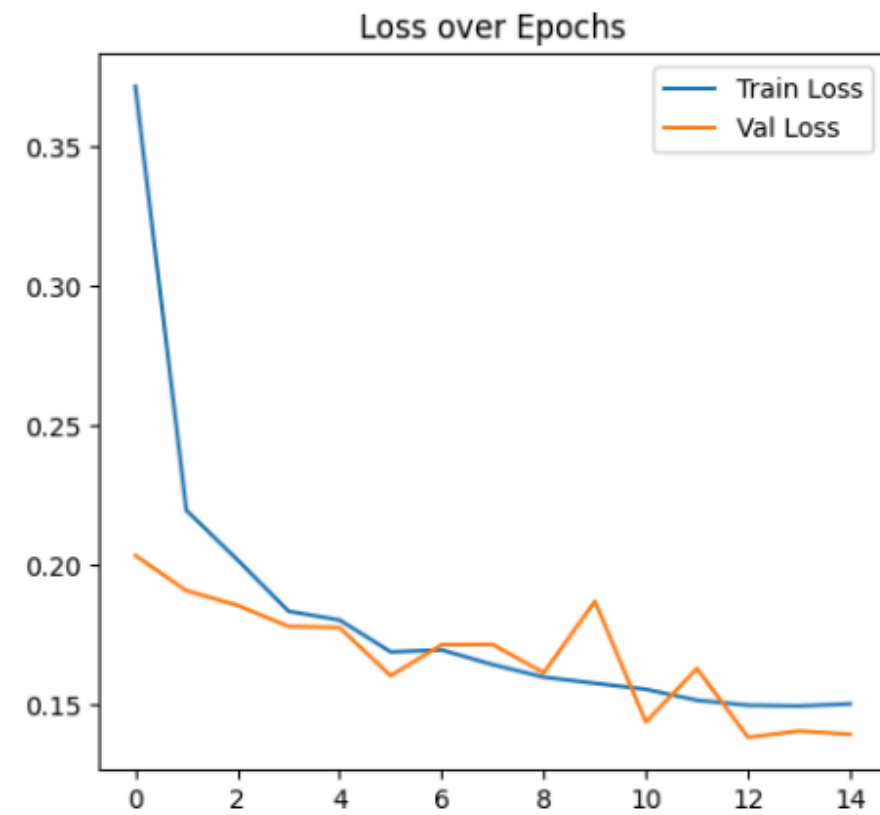


Worst Case Analysis

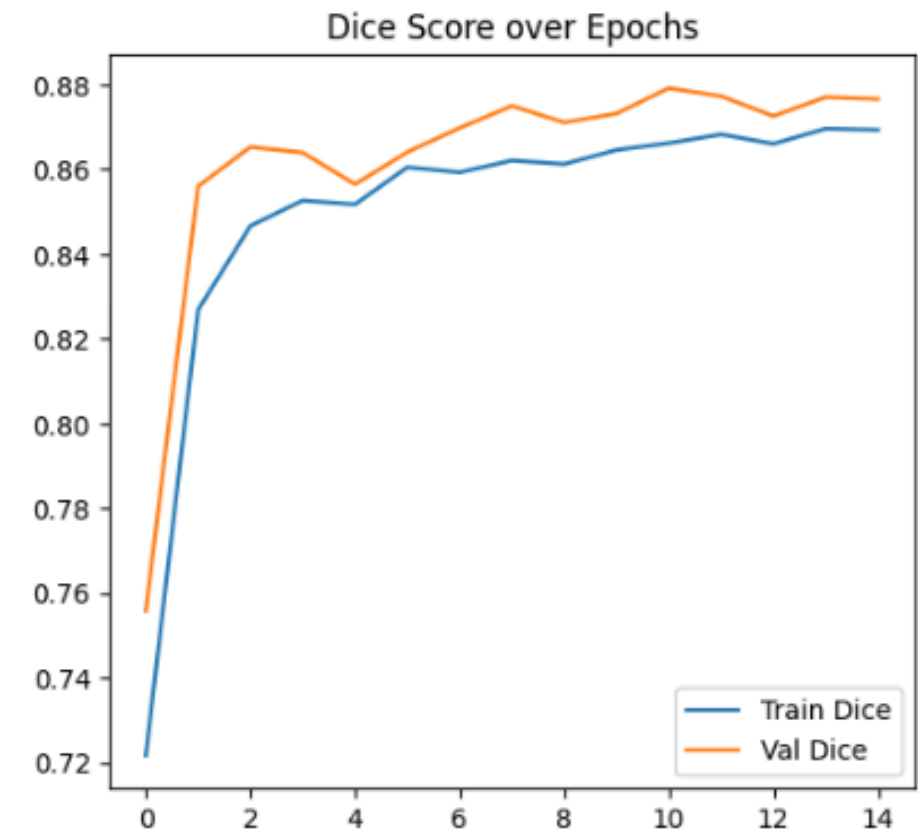
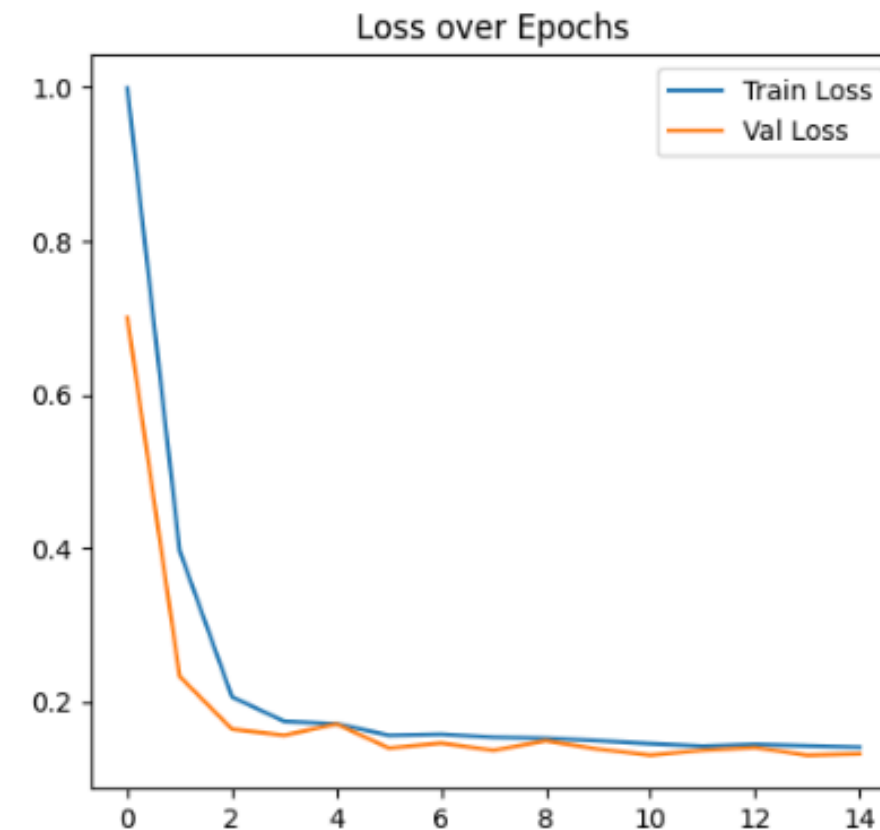
- **Severe Class Imbalance:** Background pixels dominate. The model found a "lazy" local minimum, predicting 0 for every pixel to easily minimize Binary Cross-Entropy (BCE) loss.
- **Thresholding Bottleneck:** Sigmoid probabilities in the tumor regions are too weak (e.g., 0.1 to 0.3) to breach the strict > 0.5 threshold, zeroing out the entire array during binarization.
- **Suboptimal Learning Rate / Dead Gradients:** Flawed LR selection ($1e-4$ vs $1e-3$) led to gradient collapse, causing ReLU activations to permanently "die."

Effect of Learning Rate

- **Volatility at $1e-3$:** The higher learning rate introduces mid-training instability, causing temporary spikes in validation loss and drops in Dice score.
- **Smooth Optimization at $1e-4$:** Lowering the learning rate to $1e-4$ eliminates these fluctuations, ensuring steady convergence and optimal performance.



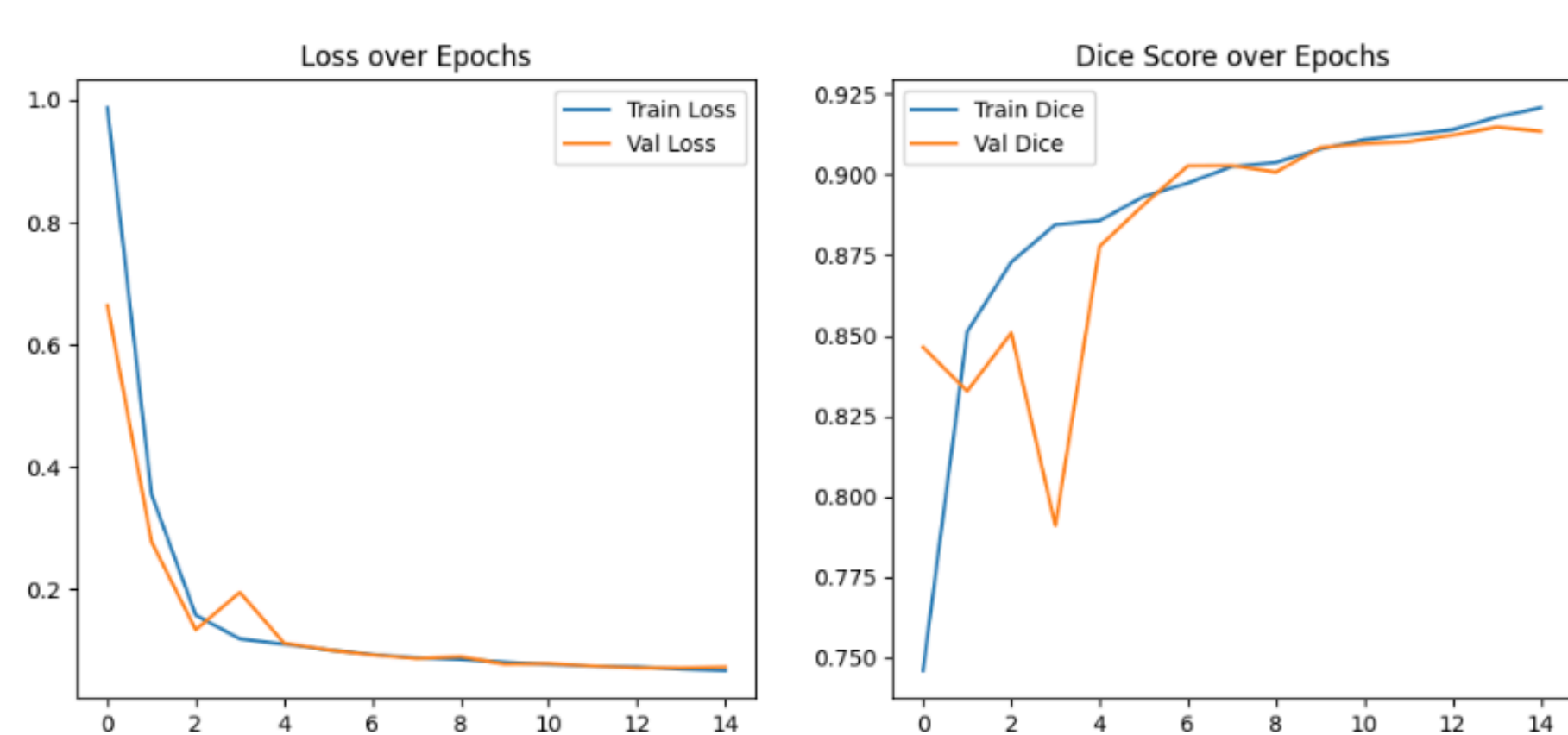
$1e-3$



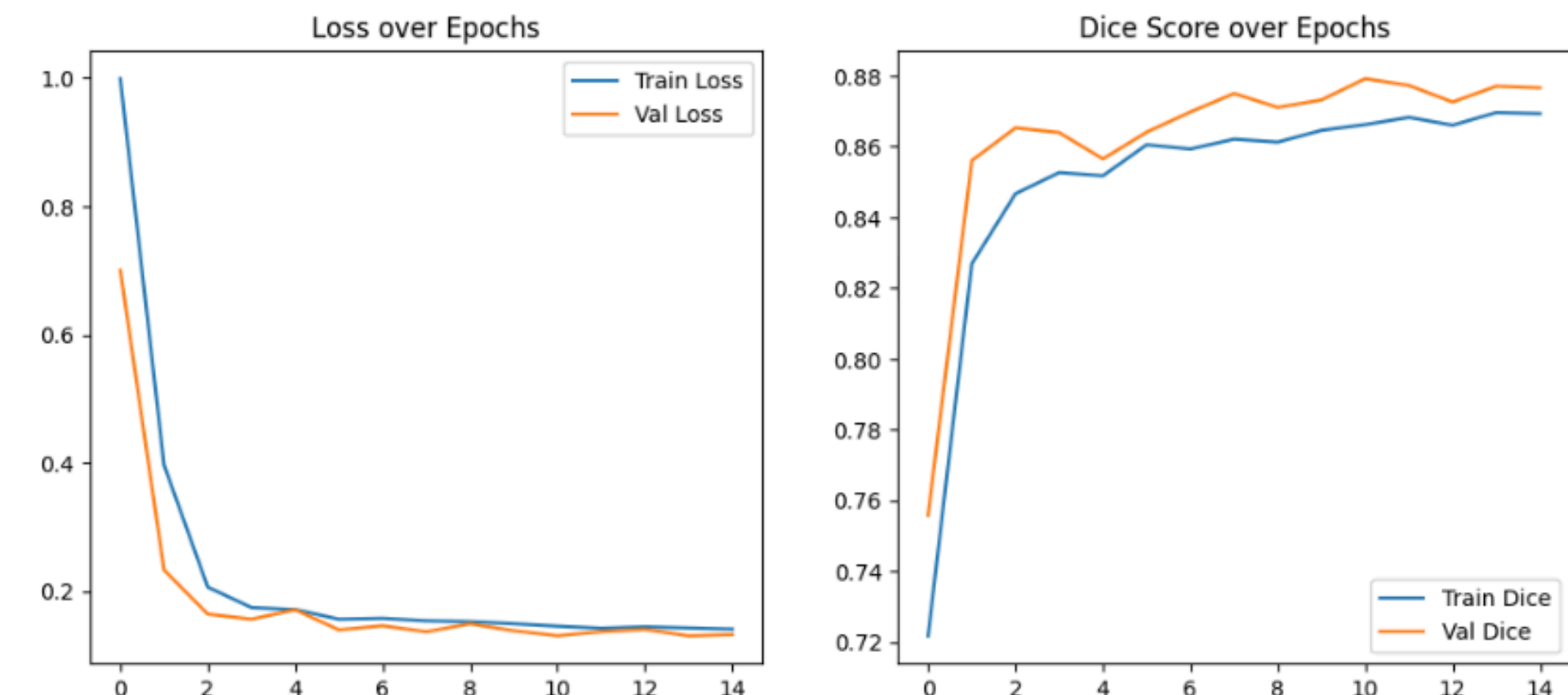
$1e-4$

Effect of Data Augmentation

Data augmentation significantly accelerated initial convergence and stabilized training within the first five epochs, although the baseline model without augmentation ultimately achieved a slightly higher final Dice score. **There is a more detailed analysis in the `G18_VA_SVA-EskiocakAliDavut.ipynb` file.**



No Augmentation



With Augmentation

Results

Configuration	Learning Rate	Data Augmentation	Test IoU (Mean ± SD)	Test Dice (Mean ± SD)
Model 1	0,0001	No	0.8863 ± 0.2359	0.9122 ± 0.2233
Model 2	0,001	Yes	0.8351 ± 0.2838	0.8691 ± 0.2701
Model 3	0,0001	Yes	0.8360 ± 0.2840	0.8694 ± 0.2709

Note: All the values in this graph produced in the Python Notebook files that I provided. You can see the values there. **No** imaginary values have been used this presentation. All of them are **real**.

Results

- The U-Net architecture is a highly successful method for detecting brain tumors in FLAIR MRI images.
- Models are mostly successful but they can struggle in some bottlenecks.
- Data Augmentation is really helpful for the success. It increases generalization.

Sources

Brain Tumor Segmentation(BraTS2020) Data Set

[Deep Learning for Computer Vision_v26.pdf](#) from Lecture Notes

Note: All the values in this graph produced in the Python Notebook files that I provided. You can see the values there. **No** imaginary values have been used this presentation. All of them are **real**.